The CourseDesk incident handover

Bash in GitHub Codespaces · about 105 minutes · Name: _____ Date: _____

Your job

You are a junior developer on the Platform day team at Larkspur College. Overnight, **coursedesk** (the course registration portal) returned errors during the late-registration window. Sam Okafor, the night on-call engineer, left a draft handover note before going home. The draft uses a retired format, guesses at the numbers and says the cause is unknown.

Your result is a **handover package** in `work/` that the next engineer can trust:

- `work/handover.txt`: a 10-line note in the current runbook format, every value backed by evidence
- `work/web-02.conf`: the faulty server's configuration, reconciled with the approved version
- `work/students-to-contact.txt`: the students whose enrolment failed, for Student Services

Each experiment answers one question about the incident. The answer becomes a line of the handover note:

Exp	Tool	Question you answer	Goes into the note as
1	`less`	Which instructions are current? How is the log laid out?	the format, `Escalation:`
2	`nano`	(You start the note and your findings file.)	lines 1 to 3
3	`cut`	When did the first 503 error happen?	`First 503:`
4	`sort`	Which request was the slowest?	`Slowest request:`
5	`uniq`	How many 503 errors were there?	`503 responses:`
6	`history`	Which server produced them?	`Faulty server:`
7	`diff`	What did the night deploy change?	`Cause:`, `Config:`
Final	all	Which students must be contacted?	`Students to contact:`

Set up (5 minutes)

Step 1. Open `https://codespaces.new/dhruvgupta-24/software-engineering-terminal-lab?quickstart=1` and create the Codespace (or resume it, if you made one before). Setup tests the lab, then the terminal prints `Setup complete`. Open a fresh terminal for the lab with `Ctrl+Shift+`` (or the ☰ menu → **Terminal** → **New Terminal**), so it picks up the lab settings.

Step 2. At the repository root, create your attempt:

```
bash lab/start.sh
```

Step 3. The last line `start.sh` prints is a `cd` command, for example `cd /workspaces/software-engineering-terminal-lab/attempts/attempt-01`. Copy and run **exactly the command it printed**: the folder number is different for each attempt. Then check that you are in the right place:

```
pwd
ls evidence work
```

The folder you are now in is your **lab root**; every experiment starts there. Full path (from `pwd`):

_____

Folder	Contents	Rule
`evidence/`	`runbook.txt`, `requests.log`, `alerts.log`, `web-02-live.conf`, `web-02-approved.conf`, `handover-draft.txt`	Original evidence. Read it, never edit it. The files are write-protected.
`work/`	`handover.txt` and `web-02.conf` (your working copies) and every file you create	Edit and save here only.

Starting again. Run `bash lab/start.sh` again from the repository root. It creates the next folder (`attempt-02`, and so on) with fresh copies and prints the `cd` command for it; your lab root is now that new folder. Earlier attempts are never changed.

How to work. Use one terminal for the whole lab, so `history` holds all your commands. Type each command yourself and read its output before moving on. Write each prediction **before** running the command. Click inside the terminal before a Ctrl shortcut (on a Mac, use Control, not Command).

> **If you get stuck.** A `:` or `(END)` at the bottom of the screen means you are inside `less`: press `q`. A bar of `^G Help ^O Write Out ...` at the bottom means you are inside `nano`: press `Ctrl+X`. A command that never finishes can be stopped with `Ctrl+C`. To see where you are, run `pwd`; to return to your lab root, `cd` to the path you wrote above.

Experiment 1 · Read the runbook with `less`

Question: which instructions are current, and how is the log laid out? · **Evidence:** `evidence/runbook.txt` · **Start in:** lab root · **Time:** 10 min

The runbook is 285 lines long, so `cat` would scroll most of it away. **`less`** shows a file one screen at a time, moves backwards as well as forwards, searches, and never changes the file. While `less` is open, your keys go to `less`, not to Bash.

Key in `less`	Action
`Space` / `b`	Forward one screen / back one screen
`g` / `G`	Jump to the first line / to the last line (`G` is Shift+g)
`/` text `Enter`	Search forward from the current screen for the text. Searches are case-sensitive.
`n` / `N`	Next match / previous match of the last search
`q`	Quit and return to the Bash prompt

1A. Move around (guided). Run `less evidence/runbook.txt`. Press `Space` twice, then `b` once. Press `G`: the last line reads `END OF RUNBOOK`. Press `g` to return to the title.

1B. Search and step between matches (guided). The runbook keeps retired sections for audit history, so the first match is not always the one to follow. Prediction: how many headings contain `NOTE FORMAT`? _____

Type `/NOTE FORMAT` and press `Enter`. You land on Section 9. Press `n` for the next match, `N` to go back once, then `n` again. Stay on the section marked **CURRENT** and read its ten lines.

Current format revision: _____ Why must the first match be rejected? _____

1C. Find two more facts (independent). Press `g` before each new search, so it starts from the top.

- Search for `ROTA`. Use `n` until you reach the CURRENT rota. Current incident lead: _____
- Search for `LOG FORMAT`. Fields in `requests.log`: status ___ time (ms) ___ endpoint ___ student ___

Press `q` to quit.

> **Watch for.** Searches are case-sensitive: `/Rota` finds nothing, because the heading says `ROTA` and the contents list says `rota`, and `less` shows `Pattern not found (press RETURN)`. The same message appears when you search from below the last match: press `Enter`, then `g`, and search again.

Keep for the handover. The format revision and the current incident lead (you write them into a file in Experiment 2), and the four field numbers above.

Experiment 2 · Edit and create files with `nano`

Goal: start the corrected note and create your findings file. · **Evidence:** `work/handover.txt` · **Start in:** lab root · **Time:** 10 min

`nano` is a text editor that runs inside the terminal. `nano FILE` opens FILE if it exists, or starts a new, empty file with that name if it does not. The two bottom lines list shortcuts: `^O` means Ctrl+O; do not type the `^` character. `M-` means Alt; this lab does not need those keys.

Key in `nano`	Action
Arrow keys, `Backspace`	Move the cursor; delete the character before it
`Ctrl+K`	Cut (delete) the whole line under the cursor
`Ctrl+O`, then `Enter`	Save ("Write Out"). nano asks `File Name to Write:` with the current name filled in; `Enter` accepts it.
`Ctrl+X`	Exit. With unsaved changes, nano asks `Save modified buffer?`: press `Y`, then `Enter`.

2A. Edit an existing file (guided). Run `cat work/handover.txt` to see the draft, then `nano work/handover.txt`. The current format begins:

```
Incident: INC-2417
Service: coursedesk
Escalation: <current incident lead>
```

Line 1 is already correct. Change line 2, `Owner: Sam Okafor`, to `Service: coursedesk`. On line 3, replace `Ravi Menon` (from the archived rota) with the incident lead you found in 1C. Leave lines 4 to 6 for the final task. Save with `Ctrl+O`, `Enter`, then exit with `Ctrl+X`.

Verify: run `cat work/handover.txt`. Then open the file in nano again and press `Ctrl+X` without changing anything. Did nano ask to save? _____ What does that tell you about the file on disk? _____

2B. Create a new file (independent). Run `nano work/findings.txt`; nano shows `[ New File ]` at the bottom. Type these two lines, replacing each `<...>` with your answer. You do not need `Enter` after the second line: nano ends the file with a newline when it saves.

```
Handover format: <revision from 1B>
Incident lead: <name from 1C>
```

Save, exit, and check with `cat work/findings.txt`. Whenever this book says **Record**, reopen this file and add a line.

Watch for. After `Ctrl+O`, press only `Enter`. Typing anything else changes the file name and saves a second file. Answering `N` to `Save modified buffer?` throws your changes away.

Keep for the handover. `work/handover.txt` (lines 1 to 3 done) and `work/findings.txt`. Check: `python3 ../../lab/check.py 2`

Experiment 3 · Select fields with `cut`

Question: when did the first 503 error happen? · **Evidence:** `evidence/requests.log` · **Start in:** lab root · **Time:** 10 min

Each line of `requests.log` is one request with six fields separated by `|`, numbered as you found in the runbook. There is no header line:

```
01:02:27|web-02|200|29|/health|-
```

`cut` prints selected fields from every line and keeps the lines in their original order.

Option	Meaning
`-d "|"`	The delimiter, the character between fields. Quote it: unquoted, Bash reads `|` as a pipe between two commands.
`-f 3` / `-f 1,2,3`	The field number(s) to keep, counting from 1. Several fields are printed with the delimiter between them.

3A. One field (guided). Predict: what will Bash do with `cut -d | -f 3 evidence/requests.log`? _____
Run it and read both error messages: Bash split the line at `|` and tried to run `cut -d` and a command called `-f`. Now run the correct form and page through the result:

```
cut -d "|" -f 3 evidence/requests.log | less
```

Press `q`. Save the same output; `>` creates the file, or replaces its contents if it already exists:

```
cut -d "|" -f 3 evidence/requests.log > work/statuses.txt
```

3B. Several fields (independent). Write a command that saves fields 1, 2 and 3 (time, server, status) of every request to `work/time-server-status.txt`. Its first two lines should be `01:00:00|web-03|200` and `01:01:04|web-01|200`. My command: _____

Now list only the 503 responses. `grep -F` searches for the exact text. Including the `|` makes it match the status field, not a number that happens to contain 503. (The runbook suggests `"|503|"` for `requests.log`; here the status is the last field, so no `|` follows it.)

```
grep -F "|503" work/time-server-status.txt | less
```

Because `cut` kept the original row order, the first line is the earliest 503. **Record** `First 503: <time>` (the note wants only the time).

Observation: run `cut -d "|" -f 5,4 evidence/requests.log | less`. Which field comes first in the output? _____
`cut` always prints fields in file order, whatever order you list them in.

Watch for. A wrong field number gives you server names or times instead of status codes; compare with runbook Section 3. Do not sort this output: the row order is what makes the first match the earliest.

Keep for the handover. The first 503 time. `work/statuses.txt` is used in Experiment 5 and your `grep` command in Experiment 6. Check: `python3 ../../lab/check.py 3`

Experiment 4 · Order lines with `sort`

Question: which request was the slowest? · **Evidence:** field 4 (response time) and field 5 (endpoint) of `requests.log` · **Start in:** lab root · **Time:** 10 min

`sort` prints the lines of its input in order. It never changes the input file. By default it compares lines as **text**, one character at a time from the left, so `10012` comes before `12`: the first characters match, and `0` comes before `2`.

Option	Meaning
`-n`	Numeric order: read the number at the start of each line and compare values
`-r`	Reverse the order. With `-n -r` the highest number comes first.

4A. Text versus numbers (guided). Predict: in text order, will the first line be the smallest response time? _____

```
cut -d "|" -f 4 evidence/requests.log | sort | less
```

Read the first lines, then press `G` for the last. Quit, then run the same pipeline with `sort -n`. Is `10093` near the top or at the bottom? Text: _____ Numeric: _____

4B. Rank and save (guided). Keep the endpoint next to each time by cutting fields 4 and 5. A line such as `10093|/api/enrol` still starts with its number, so `sort -n` can rank it:

```
cut -d "|" -f 4,5 evidence/requests.log | sort -n -r > work/latency-ranked.txt
less work/latency-ranked.txt
```

The first line is the slowest request. **Record** `Slowest request: <ms> ms on <endpoint>`.

4C. Never redirect onto the input (guided). Use a copy you can afford to lose. Predict what `cat` will show: _____

```
cp work/statuses.txt work/scratch.txt
sort work/scratch.txt > work/scratch.txt
cat work/scratch.txt
```

Observed: _____ The shell sets up `> work/scratch.txt` **before** `sort` starts, and that empties the file. `sort` then reads an empty file. Always send output to a different file from the one you read. (Try `sort evidence/alerts.log > evidence/alerts.log`: it fails with `Permission denied`, which is why the evidence is write-protected.) Remove the copy with `rm work/scratch.txt`.

> **Watch for.** Without `-n`, `sort -r` puts `953` above `10093`, because it compares the first characters `9` and `1`.

> **Keep for the handover.** The slowest request and `work/latency-ranked.txt`. Check: `python3 ../../lab/check.py 4`

Experiment 5 · Group repeated lines with `uniq`

Question: how many 503 errors were there, and which alerts fired? · **Evidence:** `evidence/alerts.log` (time | alert | server), `work/statuses.txt` · **Start in:** lab root · **Time:** 12 min

`uniq` drops a line when it is identical to the line **directly above it**; it only ever compares neighbours. `uniq -c` prints each group of identical neighbours once, with the size of the group in front. The pager repeats an alert every minute, so `alerts.log` is full of repeats.

5A. Adjacent repeats (guided). Run `cut -d "|" -f 2,3 evidence/alerts.log` and look at the 14 lines. Predict how many lines `uniq` will print: _____

```
cut -d "|" -f 2,3 evidence/alerts.log | uniq
```

Observed: _____ lines. Name a pair that still appears more than once: _____ It survived because its repeats were not next to each other.

5B. Sort first (guided). `sort` puts identical lines next to each other, so `uniq` can then keep exactly one of each:

```
cut -d "|" -f 2,3 evidence/alerts.log | sort | uniq > work/alert-types.txt
```

5C. Count, and see why order matters (guided). Run both and compare:

```
cut -d "|" -f 2,3 evidence/alerts.log | uniq -c
cut -d "|" -f 2,3 evidence/alerts.log | sort | uniq -c > work/alert-counts.txt
cat work/alert-counts.txt
```

Each output line is a count, a space, then the text. `uniq -c` pads the count with spaces on the left, which is normal:

```
      6 HighErrorRate|web-02
```

The same commands in a different order give a different answer: without `sort`, `HighErrorRate|web-02` is split into three small groups and none of them shows its total. To rank the alerts, sort the counts as numbers: `sort -n -r work/alert-counts.txt`.

5D. Count every status (independent). From `work/statuses.txt`, save one count per status code to `work/status-counts.txt`. My command: _____ **Record** `503 responses: <count>`.

> **Watch for.** If the same text appears on two lines of a `uniq -c` result, the input was not sorted.

> **Keep for the handover.** The total number of 503 responses. Check: `python3 ../../lab/check.py 5`

Experiment 6 · Find and reuse commands with `history`

Question: which server produced the 503 errors? · **Evidence:** your own commands and `work/time-server-status.txt` · **Start in:** lab root · **Time:** 10 min

Bash remembers the commands you type. `history` lists them, numbered; `history 10` shows the last ten. **Reverse search** (`Ctrl+R`) finds an earlier command when you type part of it, so you can reuse a long pipeline instead of retyping it.

Keys at the Bash prompt	Action
`Ctrl+R`, then text	Show the most recent command containing that text
`Ctrl+R` again	Step to the next older match
`Right Arrow`	Put the match on the prompt for editing, **without running it**
`Enter`	Run the match (or your edited line)
`Ctrl+G`	Cancel the search; the prompt is empty again and nothing runs

6A. List and filter (guided). Run `history 10`. Each line is a number and a command; your numbers will differ from a classmate's:

```
   37  cut -d "|" -f 3 evidence/requests.log > work/statuses.txt
```

Then filter the whole list:

```
history | grep -F "uniq"
```

The last line is usually this `history | grep` command itself, because it also contains `uniq`. Number of the command that created `work/alert-counts.txt`: _____

6B. Search and cancel (guided). At an empty prompt press `Ctrl+R` and type `uniq -c`. The prompt changes to `(reverse-i-search)`uniq -c':` followed by your most recent match. Press `Ctrl+R` again to step to an older match. Press `Ctrl+G`: the prompt is empty and nothing ran.

6C. Edit an old command (independent). Press `Ctrl+R` and type `|503`. Stop when the match is the `grep` command from 3B, `grep -F "|503" work/time-server-status.txt | less`. Press `Right Arrow`, then `End` (or `Ctrl+E`) to reach the end of the line. Press `Backspace` six times to delete `| less`, then add a pipeline that keeps only the server field, counts each server and ranks the counts highest first, saving to `work/errors-by-server.txt`. Read the whole line before pressing `Enter`.

My edited command: _____

Run `cat work/errors-by-server.txt`. **Record** `Faulty server: <server with the most 503 responses>`.

> **Watch for.** `Enter` instead of `Right Arrow` runs the old command straight away; here that only opens `less` again, so press `q` and search again. `(failed reverse-i-search)` means no earlier command contains that text: check the spelling or press `Ctrl+G`. If `Ctrl+R` does nothing at all, click inside the terminal; if it still fails, use `history | grep` and tell your instructor.

> **Keep for the handover.** The faulty server. Check: `python3 ../../lab/check.py 6`

Experiment 7 · Compare and repair with `diff`

Question: what did deploy D-118 change on web-02? · **Evidence:** `evidence/web-02-live.conf`, `evidence/web-02-approved.conf`, `work/web-02.conf` · **Start in:** lab root · **Time:** 12 min

`diff FIRST SECOND` compares two files line by line and prints the edits that would turn FIRST into SECOND. **No output means the files are identical.** When files differ, the output is the answer you asked for, not an error.

7A. Identical files (guided). Your working copy started as a copy of the live file. Run `diff evidence/web-02-live.conf work/web-02.conf`. Observed: _____

7B. Changed files (guided). Compare the live file (first) with the approved file (second):

```
diff evidence/web-02-live.conf evidence/web-02-approved.conf
```

You will see	Meaning
`7c7`	**Change** line 7 of FIRST into line 7 of SECOND
`9d8`	**Delete** line 9 of FIRST; the files then line up again after line 8 of SECOND
`10a10`	After line 10 of FIRST, **add** line 10 of SECOND
`<` line / `>` line	A line as it is in FIRST (live) / as it is in SECOND (approved)
`---`	Separates the old and new versions inside one change

Setting that was changed, from what to what: _____ Line the deploy added: _____ Approved line missing from the live file: _____

Prediction: if you swap the two file names, what happens to `<`, `>`, `a` and `d`? _____ Try it.

Save the comparison in its normal order (live first):

```
diff evidence/web-02-live.conf evidence/web-02-approved.conf > work/config-changes.diff
```

7C. Repair and verify (independent). Open `work/web-02.conf` in nano and make the three edits the comparison describes. `Ctrl+K` deletes the line under the cursor. `10a10` says where the missing line goes: after line 10 of the live file (`cache.enabled = true`). Put the cursor at the end of that line, press `Enter`, and type. Save, exit, then verify:

```
diff work/web-02.conf evidence/web-02-approved.conf
```

Repeat until this prints nothing. **Record** `Cause: <setting> changed from <approved value> to <live value>`.

> **Watch for.** `No such file or directory` means a file name is wrong; it is not a difference report. Always repair the copy in `work/`. If nano shows `[ File 'evidence/...' is unwritable ]`, you opened an original: press `Ctrl+X`, and `N` if nano asks to save.

> **Keep for the handover.** The cause, `work/config-changes.diff` and the repaired `work/web-02.conf`. Check: `python3 ../../lab/check.py 7`

Final task · Complete the handover package

Evidence: runbook Sections 11, 12 and 14, `requests.log`, your `work/findings.txt` · **Start in:** lab root · **Time:** 15 min

This task gives requirements, not commands. Several short commands with intermediate files in `work/` are as good as one long pipeline.

1. **Read the rules.** In `less`, read runbook Section 12 (student communications) and Section 11 (why a 504 is treated differently from a 503).
2. **Students to contact.** Create `work/students-to-contact.txt` exactly as Section 12 requires. Status and endpoint are separate fields, so you may need two `grep -F` commands in a row. First, plan the order of `grep`, `cut`, `sort` and `uniq`, and say why `sort` must come before `uniq`: _____
3. **Handover note.** Edit `work/handover.txt` so it contains exactly the ten lines of the rev 4 format, in order, each value taken from your findings. Delete the old rev 3 lines.
4. **Verify.** Compare `work/web-02.conf` with the approved file once more, `cat` your note, then run the full checker. Expected: `14 of 14 checks passed`.

Students to contact: _____ 503 responses: _____ Faulty server: _____ Cause: _____

Validation (10 minutes)

From your lab root:

```
python3 ../../lab/check.py          # every check
python3 ../../lab/check.py 5        # one experiment: 2 to 7, or final
```

The checker confirms that every file in `evidence/` still matches the original and checks the content, line order and structure of each required file in `work/`. Each check prints `PASS` or `FAIL` with a reason; the last line gives the score. It **never changes your files**, so run it as often as you like. If the evidence check fails, start a new attempt (see Set up) and redo the work there.

See a failing check, then fix it (do this once). Make a deliberate mistake and check it:

```
cut -d "|" -f 2,3 evidence/alerts.log | uniq > work/alert-types.txt
python3 ../../lab/check.py 5
```

Expected: `FAIL ... still has repeated lines. uniq removes only adjacent repeats; sort first, then uniq.` Press `Ctrl+R` and type `sort | uniq > work/alert` to recall your correct 5B command, press `Enter` to run it, and check again. Expected: `PASS`.

When is diff output expected? When you compare two different versions (7B, `work/config-changes.diff`), the differences are the result. When you check a copy or a repair (7A, 7C), the correct result is no output.

Skills sign-off. The checker cannot see how you used interactive tools. Your instructor watches you do each row, on these files or on a variation they choose.

Tool	Show this	Instructor
`history`	`history 10`; `history | grep -F` a word; `Ctrl+R` to a match and `Ctrl+R` to an older one; `Right Arrow`, edit, explain the line, then `Enter`; `Ctrl+G` to cancel a search	_____
`nano`	Open `work/findings.txt`, add a line, `Ctrl+O` + `Enter`, `Ctrl+X`; reopen to show the change was saved; create a new file	_____
`less`	In `evidence/runbook.txt`: `Space`, `b`, `G`, `g`; `/` search, `n` and `N` to a CURRENT section; `q`	_____

Checker score: _____ / 14

When something goes wrong

You see	It means	Do this
`-f: command not found`	The `|` after `-d` was not quoted, so Bash split the command	Write `-d "|"`
`cut` prints whole lines unchanged	No `-d`: `cut` splits at Tab characters by default	Add `-d "|"`
`cut: you must specify a list of ...`	No `-f`	Add `-f` and field numbers
`No such file or directory`	Wrong file name, or you are not in your lab root	Check the name; run `pwd`
`Permission denied`	You tried to write into `evidence/`	Send output to `work/`
An empty file after `>`	The same file was the input and the destination	Rerun with a different destination
`Pattern not found (press RETURN)`	`less` search: wrong case, or no match below this point	`Enter`, then `g`, and search again
`(failed reverse-i-search)`	No earlier command contains that text	`Ctrl+G`, then `history | grep -F`
`[ File '...' is unwritable ]`	nano opened an original in `evidence/`	`Ctrl+X` (answer `N`), open the `work/` copy
`Run this checker from inside your lab attempt`	The checker was run outside your lab root	`cd` to your lab root first

Command reference

Command or keys	Purpose
`less FILE` · `Space` `b` `g` `G` `/text` `n` `N` `q`	Read a long file: page, jump, search, quit
`nano FILE` · `Ctrl+O` `Enter` · `Ctrl+X` · `Ctrl+K`	Edit or create a file: save, exit, cut a line
`cut -d "|" -f 2` · `cut -d "|" -f 1,3`	Keep field 2, or fields 1 and 3, of every line, in row order
`sort` · `sort -n` · `sort -n -r`	Text order · numeric order · highest number first
`sort FILE | uniq` · `sort FILE | uniq -c`	One line per distinct value · count each value
`diff FIRST SECOND` · `diff A B > FILE`	Compare (`<` FIRST, `>` SECOND, `a` `c` `d`) · save the comparison
`history 10` · `history | grep -F "text"`	Recent commands · search the list
`Ctrl+R` · `Ctrl+R` · `Right Arrow` · `Enter` · `Ctrl+G`	Search history · older match · edit · run · cancel
`A | B` · `A > FILE`	Send A's output to B · save output (replaces FILE; never the input file)